

Performance Bounds — Kapsamlı Ders Notu

(4 Mayıs)

1 Performans Analizi İçin Model Tanımlama

Bir bilgisayar sisteminin performansını analiz etmek için önce bir **model** oluşturuyoruz. Bu süreç şu adımlardan oluşur:

Mevcut Sistem / Sistem Tasarımı → Kuyruk Ağı (Queuing Network) Oluştur → Modeli Parametrize Et → Modeli Değerlendir → **Performans İndeksleri Elde Et**

Analoji: Bir hastane düşün. Doktorlar, hemşireler, laboratuvar cihazları birer "servis merkezi"dir. Hastalar ise sistemdeki "müşteriler"dir. Hastanenin ne kadar verimli çalıştığını anlamak için bu sistemi modelliyoruz.

Performans indeksleri nelerdir? Sistemin ne kadar hızlı iş yaptığı (throughput = X) ve müşterilerin ne kadar beklediği (response time = R) gibi ölçümlerdir.

=Tanım: Performance Model

Gerçek bir sistemin matematiksel soyutlamasıdır. Sistemi kuyruklar ve servis merkezleri olarak temsil eder.

2 Performance Bounds Nedir?

Performance Bounds (Performans Sınırları), bir sistemin performansının alabileceği **en iyi** ve **en kötü** değerleri hızlıca tahmin etmemizi sağlayan bir tekniktir.

Neden kullanıyoruz?

- Sistemin **darboğazını (bottleneck)** hızla tespit eder.
- Kalem-kağıtla bile hesaplanabilir, çok hızlıdır.
- Birden fazla sistem alternatifini aynı anda karşılaştırabilirsin.
- Tam ve detaylı simülasyon yapmadan önce **ilk kesme (first cut)** analiz tekniği olarak kullanılır.

Analoji: Bir arabaya binmeden önce "Bu yolda kaç dakikada giderim?" diye düşünürsün. GPS'e bakmadan bile "en az 20, en fazla 60 dakika" diyebilirsin. İşte performance bounds bu ön tahmin gibidir.

=Main Takeaway

Performance bounds, tam model çözmekten çok daha hızlıdır ve sistem darboğazını anlamanızı sağlar.

3 Bounding Analysis — Temel Kavramlar

Bounding Analysis, X (throughput) ve R (response time) performans indeksleri için **üst ve alt sınırları** belirler. Bu sınırlar iki değişkene göre fonksiyon olarak ifade edilir:

- N = Sistemdeki kullanıcı sayısı (kapalı modeller için)
- λ (lambda) = İş yükü varış hızı (açık modeller için)

İki tür sınır vardır:

1. **Optimistic (İyimser):** X için üst sınır, R için alt sınır — "en iyi ne olabilir?"
2. **Pessimistic (Kötümser):** X için alt sınır, R için üst sınır — "en kötü ne olabilir?"

=Önemli Varsayım

Bir müşterinin bir merkezde ne kadar servis alacağı (service demand), sistemde kaç başka müşteri olduğuna **BAĞLI DEĞİLDİR**. Bu, analizimizi basitleştirir.

4 Bottleneck (Darboğaz) Kavramı

=Tanım: Bottleneck

Bir sistemdeki servis merkezleri arasında **en yüksek servis talebine** sahip olan kaynaktır. Matematiksel olarak:

$$D_{max} = \max_k \{D_k\}$$

Bottleneck neden önemlidir?

- Tüm sistemin maksimum performansını sınırlar.
- En yüksek kullanım oranına (utilization) sahip kaynaktır.
- Sistem doyuma (saturation) ilk bu kaynak ulaşır.

Trafik Analojisi: Geniş bir yoldan dar bir yola geçerken tüm arabalar yavaşlar. İşte o dar geçit sistemin darboğazıdır. Diğer şeritleri ne kadar genişletirsen genişlet, o dar geçidi çözmeden trafik akışı iyileşmez.

Pratik Örnek: Bir web sunucusunda CPU, Disk1, Disk2 varsa ve CPU en çok zaman harcanan yer ise → CPU bottleneck'tir. Diski ne kadar hızlandırırsan hızlandır, sistem performansı CPU nedeniyle sınırlı kalır.

Bounding Analysis'in Faydaları:

- Darboğazı tespit eder ve etkisini sayısal olarak gösterir.
- **System Sizing** (sistem boyutlandırma) için çok kullanışlıdır — hangi CPU/disk konfigürasyonu seçmeliyim?
- **System Upgrades** (sistem güncelleme) kararlarında yol gösterir.

Sınav Odak Noktası (Exam Focus)

Bottleneck = D_{max} olan kaynak = en yüksek utilization'a sahip kaynak. Bu üç ifade birbirine eşdeğerdir.

5 Notasyon — Tüm Değişkenler

Formüllerde kullanılacak tüm sembolleri bu tabloda öğrenelim:

Sembol	Anlamı
K	Servis merkezi sayısı
D_k	k . merkezin servis talebi (service demand)
D	Tüm merkezlerin toplam servis talebi ($\sum D_k$)
D_{max}	Herhangi bir merkezin maksimum servis talebi (bottleneck'in D değeri)
Z	Ortalama "think time" (kullanıcının sonraki isteği göndermeden önceki bekleme süresi)
X	Sistem throughput'u (birim zamanda tamamlanan iş sayısı)
R	Sistem response time'ı (bir işin sistemde geçirdiği toplam süre)

- **Service Demand (D_k):** Bir müşterinin k . merkezde harcadığı toplam ortalama servis süresidir. Formül: $D_k = V_k \times S_k$ (V_k = ziyaret sayısı, S_k = her ziyarette ortalama servis süresi).
- **Think Time (Z):** İnteraktif sistemlerde kullanıcının bir işlem tamamlandıktan sonra bir sonraki isteği göndermeden önce "düşündüğü" süredir. (Örn: web sayfasını okuma süresi).

6 Asimptotik Sınırlar — Genel Mantık

Asymptotic Bounds (Asimptotik Sınırlar), iki uç yük koşulunu inceleyerek türetilir:

- **Light Load (Hafif Yük):** Sistemde çok az müşteri var $\rightarrow$ kuyruk yok $\rightarrow$ bekleme yok $\rightarrow$ en iyi durum.
- **Heavy Load (Ağır Yük):** Sistemde çok fazla müşteri var $\rightarrow$ kuyruklar uzuyor $\rightarrow$ bottleneck doyuma ulaşıyor $\rightarrow$ en kötü durum.

Bu iki uç koşuldan elde edilenler:

- **İyimser sınır:** X üst sınırı + R alt sınırı (en iyi performans)
- **Kötümser sınır:** X alt sınırı + R üst sınırı (en kötü performans)

Analoji: Bir restoranın kapasitesini düşün. Sabah 10'da 2 müşteri varsa herkes hemen yemek alır (light load). Akşam 7'de 100 kişi varsa herkes bekler (heavy load). Performans sınırları bu iki uç durumu modelliyor.

7 Açık Modeller (Open Models) — X ve R Sınırları

=Tanım: Open Model

Dışarıdan sisteme sürekli iş gelen modeldir. Müşteri sayısı sabit değil, λ (varış hızı) sabittir. Örn: bir web sunucusuna gelen HTTP istekleri.

X Sınırı (Throughput)

Açık modellerde asıl soru şudur: "Sistem ne kadar yük kaldırabilir?" Eğer $\lambda > \lambda_{sat}$ ise sistem **SATURE** olur $\rightarrow$ yeni işler sonsuz bekleme yaşar.

Türetme:

- Utilization formülü: $U_k = X \times D_k$
- Hiçbir kaynak %100'ün üzerinde kullanılamaz: $U_k \leq 1$
- En kısıtlayıcı olan bottleneck: $X \times D_{max} \leq 1$

Bu şunun anlamına gelir:

$$\lambda_{sat} = \frac{1}{D_{max}}$$

Örnek: $D_{max} = 0.5$ saniye ise $\rightarrow \lambda_{sat} = 1/0.5 = 2$ iş/saniye. Sisteme saniyede 2'den fazla iş gelirse sistem sature olur. Dolayısıyla:

$$X(\lambda) \leq \frac{1}{D_{max}}$$

R Sınırı (Response Time)

Alt sınır (optimistic): En iyi durumda hiçbir müşteri birbirini beklemes (kuyruk yok). Bu durumda response time sadece tüm servislerin toplamına eşittir:

$$R(\lambda) \geq D = \sum_k D_k$$

Üst sınır (pessimistic): Açık modellerde R için **üst sınır yoktur!** Bunu anlamak önemli: λ sabit kalsa bile işler "toplu" gelebilir. Örneğin her n/λ saniyede n iş aynı anda gelirse $\rightarrow$ ortalama varış hızı yine λ 'dır, ama sona gelen müşteriler öndeki herkesi beklemek zorundadır. N arttıkça bekleme süreleri sonsuza gider.

=Kritik Sonuç

Açık modellerde, λ ne kadar küçük olursa olsun, R için bir üst sınır **YOKTUR**. Gerçek sistemlerde bile bu geçerlidir — burst (ani yük) trafiği sistemi mahvedebilir.

Özet — Açık Model Sınırları

$$X(\lambda) \leq \frac{1}{D_{max}} \quad \text{ve} \quad R(\lambda) \geq D$$

Taralı geçerli performans bölgesine göre: R en az D olabilir, X en fazla $1/D_{max}$ olabilir. Throughput bottleneck tarafından sınırlıdır. Response time için sadece alt sınır bulunur.

8 Kapalı Modeller (Closed Models) — X Sınırları

=Tanım: Closed Model

Sistemde sabit sayıda N müşteri döngü halinde dolaşır. Dışarıdan yeni müşteri gelmez.
Örn: N terminali olan interaktif bir sistem.

Kapalı modellerde önce X sınırlarını bulup, sonra bunları **Little's Law** ile R sınırlarına dönüştürürüz.

$$N = X \times (R + Z)$$

Light Load — Alt Sınır

1 Müşteri Durumu: $N = 1$ için Little's Law uygularsak:

$$1 = X(D + Z) \implies X = \frac{1}{D + Z}$$

Bu, X için alt sınırı verir (en kötü throughput) çünkü 1 müşteri ile sistem hiç boş kalmaz ama kuyruk da oluşmaz.

N Müşteri Ekleme (En Kötü Durum): Yeni gelen her iş, önceki herkesin arkasına kuyruğa girer (Örneğin tüm müşterilerin aynı anda farklı istasyonlarda ya da tek bir istasyonda beklemesi). Bu durumda X :

$$X = \frac{N}{ND + Z}$$

$N \rightarrow \infty$ iken limiti $1/D$ 'ye yaklaşır (bottleneck **DEĞİL**, toplam D 'ye dikkat!).

Light Load — Üst Sınır

En iyi durum: her gelen iş boş kuyruk bulur, hemen servis alır.

$$X = \frac{N}{D + Z}$$

Bu ifade N ile doğrusal büyür — ama tabii ki bottleneck sınırını aşamaz.

Heavy Load — Üst Sınır

Sistemde çok fazla müşteri olduğunda, ilk doyuma ulaşan kaynak bottleneck'tir.

$$U_k(N) = X(N) \times D_k \leq 1$$

En kısıtlayıcı olan D_{max} için mutlak üst sınır:

$$X(N) \leq \frac{1}{D_{max}}$$

X(N) için Tam Sınır Formülü

Hem alt hem üst sınırları birleştiriyoruz:

$$\boxed{\frac{N}{ND + Z} \leq X(N) \leq \min\left(\frac{1}{D_{max}}, \frac{N}{D + Z}\right)}$$

N^* Nedir (Kırılma Noktası)? N^* , iki üst sınırdan hangisinin daha bağlayıcı (küçük) olduğunu belirleyen kritik müşteri sayısıdır:

$$\boxed{N^* = \frac{D + Z}{D_{max}}}$$

- $N < N^*$ ise: Sistem hafif yükte $\rightarrow$ üst sınır $\frac{N}{D+Z}$ (light load domine eder)
- $N > N^*$ ise: Sistem ağır yükte $\rightarrow$ üst sınır $\frac{1}{D_{max}}$ (bottleneck domine eder)

Analoji: N^* bir "kırılma noktası"dır. Bu noktaya kadar daha fazla kullanıcı ekledikçe throughput artar. Bu noktadan sonra bottleneck tıkanmış, throughput artık artmaz.

9 Kapalı Modeller — R(N) Sınırları

$X(N)$ sınırlarını bulduktan sonra Little's Law ($X(N) = \frac{N}{R(N)+Z}$) ile $R(N)$ sınırlarına dönüştürüyoruz:

$$\frac{N}{ND + Z} \leq \frac{N}{R(N) + Z} \leq \min\left(\frac{1}{D_{max}}, \frac{N}{D + Z}\right)$$

Üyeleri ters çevirip düzenleyince elde edilen nihai formül:

$$\boxed{\max(D, N \cdot D_{max} - Z) \leq R(N) \leq ND}$$

Bu formülü parça parça anlayalım:

- **Üst sınır (ND):** En kötü durum — her müşteri her istasyonda tüm diğer $N-1$ müşteriyi bekler. Response time N ile lineer büyür.
- **Alt sınır ($\max(D, N \cdot D_{max} - Z)$):**
 - D : Sıfır kuyruk — en az toplam servis süresi kadar beklenir.
 - $N \cdot D_{max} - Z$: Heavy load koşulu — bottleneck doyuma ulaştığında response time bu ifadeyle alt sınırlanır.
- **$\max()$ Neden var?** N küçükken D daha büyük, N büyükken $N \cdot D_{max} - Z$ daha büyük olur. Her zaman ikisinden büyük olanı alt sınır olur.

10 Özet — Tüm Sınırlar Bir Arada

Bu tablolar her şeyi özetliyor. Ezberlemen gereken temel formüller:

Not: $Z = 0$ (batch sistem) olduğunda sınırlar daha sıkı, $Z > 0$ (interaktif sistem) olduğunda daha geniştir.

Model Türü	Throughput (X) Sınırları	Response Time (R) Sınırları
Açık Modeller	$X(\lambda) \leq \frac{1}{D_{max}}$	$R(\lambda) \geq D$
Kapalı Modeller	$\frac{N}{ND+Z} \leq X(N) \leq \min\left(\frac{N}{D+Z}, \frac{1}{D_{max}}\right)$	$\max(D, ND_{max} - Z) \leq R(N) \leq ND$

11 Örnek Sisteme Giriş

Sistem: 1 CPU + 2 Disk + N Terminal (interaktif kullanıcılar)

Parametre	Değer	Parametre	Değer
D_1 (CPU)	2.0 s	V_2 (Disk 2 ziyareti)	10
D_2 (Disk 2)	0.5 s	V_3 (Disk 3 ziyareti)	100
D_3 (Disk 3)	3.0 s	S_2 (Disk 2 servis)	0.05 s
Z (Think time)	15 s	S_3 (Disk 3 servis)	0.03 s

Adım 1: Kontrol ($D_k = V_k \times S_k$)

- $D_2 = 10 \times 0.05 = 0.5 \checkmark$
- $D_3 = 100 \times 0.03 = 3.0 \checkmark$
- Toplam $D = 2.0 + 0.5 + 3.0 = 5.5$ s

Adım 2: Bottleneck Kim?

$$D_{max} = \max(2.0, 0.5, 3.0) = 3.0 \text{ s} \implies \text{Disk 3 bottleneck!}$$

Adım 3: Kırılma Noktası (N^*)

$$N^* = \frac{D + Z}{D_{max}} = \frac{5.5 + 15}{3.0} = \frac{20.5}{3.0} \approx 6.83 \approx 7$$

Adım 4: Sınırları Belirleme

- $X(N)$ için:

$$\frac{N}{5.5N + 15} \leq X(N) \leq \min\left(\frac{N}{20.5}, 0.333\right)$$

- $R(N)$ için:

$$\max(5.5, 3N - 15) \leq R(N) \leq 5.5N$$

Bu denklemler kullanılarak sistem alternatifleri kolaylıkla birbirleriyle karşılaştırılabilir.

Ana Çıkarımlar (Main Takeaways)

- Performance bounds, tam analiz yapmadan sistemin davranışını tahmin etmemizi sağlar.
- **Bottleneck** (D_{max}) her zaman sistemin "tavan"ını belirler.
- Açık modellerde R için üst sınır **YOKTUR**; burst trafik sistemi çökertebilir.
- Kapalı modellerde N^* noktasına kadar kullanıcı eklemek throughput'u artırır. Sonrasında bottleneck tıkanmış olduğundan throughput artmaz, sadece response time büyür.

Sınav Odak Noktası (Exam Focus)

- **Bottleneck** = D_{max} olan kaynak = en yüksek utilization'a sahip kaynak = sistemi sınırlayan kaynak.
- **Açık Model Sınırları:** $X \leq 1/D_{max}$ ve $R \geq D$.
- **Kapalı Model Sınırları:**
 - X : $N/(ND + Z) \leq X(N) \leq \min(N/(D + Z), 1/D_{max})$
 - R : $\max(D, ND_{max} - Z) \leq R(N) \leq ND$
- **Kırılma Noktası:** $N^* = (D + Z)/D_{max}$ formülünü mutlaka bil! ($N < N^*$ ise light load, $N > N^*$ ise heavy load / bottleneck doymuş).
- $D_k = V_k \times S_k$ formülünü bilmeli ve servis taleplerini hesaplayabilmelisin.
- **Little's Law:** $N = X(R + Z)$ — closed model için X ve R arasında geçiş yapmanı sağlar.

12 Orijinal Sistem — Hesaplamalar ve Grafik

Önceki bölümde sistemimizi tanımladık. Şimdi formülleri uygulayalım.

Sistem parametreleri:

$$D_1 = 2.0s, \quad D_2 = 0.5s, \quad D_3 = 3.0s$$

$$D = 5.5s, \quad D_{max} = 3.0s \text{ (Disk 3)}, \quad Z = 15s$$

$X(N)$ Sınırları:

$$\frac{N}{N \cdot 5.5 + 15} \leq X(N) \leq \min\left(\frac{N}{20.5}, \frac{1}{3}\right)$$

$R(N)$ Sınırları:

$$\max(5.5, 3N - 15) \leq R(N) \leq 5.5N$$

Alt sınırın $\max()$ içindeki iki terimini inceleyelim:

- $D = 5.5$: Sabit alt sınır — kuyruk olmasa bile en az bu kadar beklenir.
- $3N - 15$: N büyüdükçe büyüyen terim — bottleneck etkisi.

Bu ikisi ne zaman eşitlenir?

$$5.5 = 3N - 15 \implies 3N = 20.5 \implies N \approx 6.83 = N^*$$

Yani $N < 7$ iken alt sınır = 5.5 (sabit), $N > 7$ iken alt sınır = $3N - 15$ (lineer artar). Bu N^* noktasıdır.

Grafikleri Yorumlama:

- **$R(N)$ Eğrisi:** $N^* = 7$ 'ye kadar response time neredeyse sabit ($D = 5.5$ seviyesinde), sonra lineer artıyor. Gerçek sistem her zaman sınırların belirlediği geçerli bölgenin içinde kalır.
- **$X(N)$ Eğrisi:** Throughput N artarken önce hızla yükseliyor, N^* sonrasında $1/D_{max} = 1/3 \approx 0.333$ tavanına yapıyor.

Ana Çıkarımlar (Main Takeaways)

Orijinal sistemde Disk 3 bottleneck'tir ($D_{max} = 3s$). Sistem $N^* \approx 7$ kullanıcıdan sonra doyuma ulaşır ve throughput 0.333 iş/saniyeyi geçemez.

13 What-If Analizi Nedir?

What-If Analizi, sistemde değişiklik yapmadan önce "şunu yapsam ne olur?" sorusunu cevaplayan matematiksel yoludur. Olası dört temel senaryo incelenir:

1. CPU'yu 2 kat hızlandır
2. Disk yüklerini dengele (load balancing)
3. İkinci hızlı disk ekle
4. Üç değişikliği birlikte uygula

Neden bu analiz değerlidir? Gerçek donanım satın almadan, sadece D değerlerini değiştirerek sistemi simüle edebilirsin. Hangi yatırımın en çok kazanç getireceğini önceden görebilirsin. *Analoji:* Bir müzik grubunun prova yapmadan önce nota kağıdı üzerinde hangi enstrümanı değiştirirsek ses daha iyi çıkar diye tartışması gibi.

14 Alternatif 1 — CPU'yu 2 Kat Hızlandır

Ne değişiyor? CPU 2 kat hızlanınca D_1 yarıya iner:

$$D_1 : 2.0s \rightarrow 1.0s$$

$$D_2 = 0.5s \text{ (değişmedi)}, \quad D_3 = 3.0s \text{ (değişmedi)}$$

Yeni hesaplamalar:

- Yeni toplam D : $D_{new} = 1.0 + 0.5 + 3.0 = 4.5s$
- Bottleneck değişti mi? $D_{max} = \max(1.0, 0.5, 3.0) = 3.0s \Rightarrow$ **Disk 3 hâlâ bottleneck!**
- Yeni N^* : $N^* = \frac{D+Z}{D_{max}} = \frac{4.5+15}{3.0} = \frac{19.5}{3} = 6.5$

Yeni Sınırlar:

$$X(N) \leq \min\left(\frac{N}{19.5}, \frac{1}{3}\right)$$
$$\max(4.5, 3N - 15) \leq R(N) \leq 4.5N$$

Sonuç ve Yorum: Eğriler Orijinal sistem ile neredeyse **aynı seyri izler!** *Neden?* Çünkü CPU zaten bottleneck değildi. $D_1 = 2.0s$ iken $D_3 = 3.0s$ 'tı — CPU'yu hızlandırman bottleneck olan Disk 3'ü hiç etkilemiyor.

- **Throughput tavanı:** Hâlâ $1/3 \approx 0.333$ (değişmedi!)
- $R(N)$ **üst sınırı:** ND yerine $4.5N$ — biraz iyileşme var ama küçük.

Bottleneck olmayan kaynağı iyileştirmek sisteme neredeyse hiç kazanç sağlamaz. Para harcamadan önce bottleneck'i tespit et!

Sınav Odak Noktası (Exam Focus)

"CPU 2x hızlandırıldı, throughput ne oldu?" → Cevap: Bottleneck Disk 3 olduğu için throughput tavanı ($1/D_{max} = 1/3$) değişmedi.

15 Alternatif 2 — Disk Yüklerini Dengele

Fikir Nedir? Disk 3 (hızlı disk, $S_3 = 0.03s$) çok fazla yük taşıyor — 100 ziyaret. Disk 2 (yavaş disk, $S_2 = 0.05s$) ise az yük taşıyor — 10 ziyaret. Bazı dosyaları Disk 3'ten Disk 2'ye taşıyarak yükü dengeleyelim. **Hedef:** $D_2 = D_3$

Matematiksel Çözüm: İki bilinmeyen (V_2, V_3), iki denklem:

1. Toplam ziyaret sayısı korunur: $V_2 + V_3 = 110$

2. Servis taleplerini eşitle: $V_2 \cdot S_2 = V_3 \cdot S_3 \implies V_2 \times 0.05 = V_3 \times 0.03$

$$V_2 = \frac{0.03}{0.05} V_3 = 0.6 \cdot V_3$$

Denklem 1'e koyarsak:

$$0.6V_3 + V_3 = 110 \implies 1.6V_3 = 110 \implies V_3 = 68.75 \approx 69$$

$$V_2 = 110 - 69 = 41$$

Yeni servis talepleri:

$$D_2 = 41 \times 0.05 = 2.05 \approx 2.06s$$

$$D_3 = 69 \times 0.03 = 2.07 \approx 2.06s \quad \checkmark$$

Yeni sistem parametreleri:

$$D_1 = 2.0s, \quad D_2 = D_3 = 2.06s$$

$$D_{new} = 2.0 + 2.06 + 2.06 = 6.12s$$

$$D_{max} = \max(2.0, 2.06, 2.06) = 2.06s \implies \text{Yeni bottleneck: her iki disk eşit!}$$

$$N^* = \frac{6.12 + 15}{2.06} = \frac{21.12}{2.06} \approx 10.25$$

Yeni throughput tavanı:

$$X_{max} = \frac{1}{D_{max}} = \frac{1}{2.06} \approx 0.485 \text{ iş/s}$$

Sonuç ve Yorum: Önceki tavan 0.333 → Yeni tavan 0.485. Bu **%46 throughput artışı** demektir! Ve bunu sadece dosyaları taşıyarak yaptık. Dikkat: D toplamı arttı ($5.5 \rightarrow 6.12$), bu yüzden $R(N)$ başlangıç değeri biraz yükseldi. Hafif yükte biraz daha yavaş, ağır yükte çok

daha iyidir. *M&M şeker analojisi*: Yüğü dengeli dağıtmak, iki kaba eşit miktarda şeker koymak gibidir; hiçbir kap gereğinden erken taşmaz.

Ana Çıkarımlar (Main Takeaways)

Load balancing (yük dengeleme) bottleneck'i ortadan kaldırabilir veya etkisini azaltabilir. Donanım yatırımı yapmadan büyük kazanım sağlayan bir tekniktir.

Sınav Odak Noktası (Exam Focus)

$D_k = V_k \times S_k$ formülüyle yük dengeleme hesabı yapabilmelisin. İki disk arasında yük dengelemek için denklem sistemi kurulur.

16 Alternatif 3 — İkinci Hızlı Disk Ekle

Fikir Nedir? Disk 3'ün yükünü tamamen ortadan kaldırmak yerine, yarı yarıya bölmek için aynı özellikte ($S_4 = 0.03s$) ikinci bir disk ekleyelim.

Ne değişiyor? Disk 3'ün 100 ziyaretinin yarısı yeni Disk 4'e gidiyor: $V_3 \rightarrow 50$, $V_4 = 50$

$$D_3 = 50 \times 0.03 = 1.5s$$

$$D_4 = 50 \times 0.03 = 1.5s$$

Sistem şimdi $K = 4$ servis merkezli:

$$D_1 = 2.0s, \quad D_2 = 0.5s, \quad D_3 = 1.5s, \quad D_4 = 1.5s$$

$$D = 2.0 + 0.5 + 1.5 + 1.5 = 5.5s \text{ (aynı kaldı!)}$$

$$D_{max} = \max(2.0, 0.5, 1.5, 1.5) = 2.0s \implies \text{CPU yeni bottleneck!}$$

$$X_{max} = \frac{1}{D_{max}} = \frac{1}{2.0} = 0.5 \text{ iş/s}$$

$$N^* = \frac{5.5 + 15}{2.0} = \frac{20.5}{2} = 10.25$$

Sonuç ve Yorum: Önceki tavan $0.333 \rightarrow$ Yeni tavan 0.5 . Bu **%50 throughput artışı** demektir! Disk 3 artık sorun değil ama şimdi CPU bottleneck oldu. Dikkat: D toplamı değişmedi ($5.5s$), bu yüzden $R(N)$ üst sınırı yine $5.5N$.

Ana Çıkarımlar (Main Takeaways)

Donanım eklemek (ikinci disk) bottleneck'i kaydırır. Sistem iyileştirmesi zincirleme bir süreçtir.

Sınav Odak Noktası (Exam Focus)

Yeni disk eklenince D toplamı değişebilir veya değişmeyebilir — dikkatli hesapla. Bottleneck'in hangi kaynağa geçtiğini bulmak önemlidir.

17 Alternatif 4 — Üç Değişikliği Birlikte Uygula

Fikir Nedir? En iyi sonucu almak için hem CPU 2x hızlandırılıyor ($D_1 = 1.0s$) hem de yük Disk 2, Disk 3 ve yeni Disk 4 arasında eşitleniyor.

Matematiksel Çözüm: Üç bilinmeyen (V_2, V_3, V_4), üç denklem:

1. $V_2 + V_3 + V_4 = 110$
2. $V_2 \cdot S_2 = V_3 \cdot S_3 \implies 0.05V_2 = 0.03V_3$
3. $V_3 \cdot S_3 = V_4 \cdot S_4 \implies 0.03V_3 = 0.03V_4 \implies V_3 = V_4$

Denklem 2'den: $V_2 = \frac{0.03}{0.05}V_3 = 0.6V_3$. Denklem 1'e yerleştirirsek:

$$0.6V_3 + V_3 + V_3 = 110 \implies 2.6V_3 = 110 \implies V_3 \approx 42.3$$

$$V_4 \approx 42.3, \quad V_2 \approx 25.4$$

Yeni D değerleri ve Bottleneck:

$$D_2 = 25.4 \times 0.05 \approx 1.27s$$

$$D_3 = D_4 = 42.3 \times 0.03 \approx 1.27s$$

$$D = 1.0 + 1.27 + 1.27 + 1.27 = 4.81s$$

$$D_{max} = \max(1.0, 1.27, 1.27, 1.27) = 1.27s \implies \text{Üç disk eşit bottleneck!}$$

$$X_{max} = \frac{1}{1.27} \approx 0.787 \text{ iş/s}$$

$$N^* = \frac{4.81 + 15}{1.27} = \frac{19.81}{1.27} \approx 15.6$$

Senaryoların Karşılaştırma Tablosu:

Senaryo	Darboğaz (D_{max})	Max Verim (X_{max})	Kırılma (N^*)
Orijinal Sistem	3.0s	0.333	~ 7
Alternatif 1 (CPU 2x)	3.0s	0.333	~ 6.5
Alternatif 2 (Disk Dengele)	2.06s	0.485	~ 10
Alternatif 3 (2. Disk Ekle)	2.0s	0.500	~ 10
Alternatif 4 (Hepsi)	1.27s	0.787	~ 16

Ana Çıkarımlar (Main Takeaways)

En büyük kazanç, tüm değişikliklerin birlikte uygulandığı Alternatif 4'te elde ediliyor. Tek bir iyileştirme yetersiz kalıyor çünkü bottleneck bir kaynaktan diğerine kayıyor.

18 What-If Analizinin Genel Dersleri

Bu dört alternatiften çıkan evrensel dersler şunlardır:

- **Ders 1 — Bottleneck olmayan kaynağı iyileştirmek işe yaramaz:** CPU'yu hızlandırdık ama D_{max} değişmediği için yatırım boşa gitti.
- **Ders 2 — Load balancing çok etkilidir:** Sadece yazılımsal yük dağılımıyla %46 artış sağlandı.
- **Ders 3 — Donanım eklemek bottleneck'i kaydırır:** İkinci disk eklendiğinde darboğaz Disk'ten CPU'ya geçti.
- **Ders 4 — En iyi sonuç için tüm bottleneck'leri çöz:** Sisteme bütünsel yaklaşım hem donanım hem konfigürasyon güncellemesi yapmak (Alt 4) performansı katlar.

GENEL ÖZET VE EXAM FOCUS — TÜM KONULAR

Tüm Formüller Bir Arada:

- **Açık Modeller:**

$$X(\lambda) \leq \frac{1}{D_{max}}, \quad R(\lambda) \geq D$$

- **Kapalı Modeller:**

$$\frac{N}{ND + Z} \leq X(N) \leq \min\left(\frac{N}{D + Z}, \frac{1}{D_{max}}\right)$$
$$\max(D, ND_{max} - Z) \leq R(N) \leq ND$$

- **Temel Denklemler:**

$$N^* = \frac{D + Z}{D_{max}}, \quad D_k = V_k \times S_k, \quad D = \sum_k D_k$$

=Sınavda Çıkabilecek Soru Tipleri

- **Tip 1 — Bottleneck bul:** D_k değerlerini hesapla, hangisi en büyükse bottleneck olur. $X_{max} = 1/D_{max}$.
- **Tip 2 — N^* hesapla:** $N^* = (D + Z)/D_{max}$. (N^* öncesi light load, sonrası heavy load).
- **Tip 3 — Sınır formülleri yaz:** $X(N)$ ve $R(N)$ için alt ve üst sınırları yaz.
- **Tip 4 — What-if analizi:** Bir kaynak değişince yeni D değerlerini hesapla, yeni bottleneck'i bul, yeni sınırları yaz, iyileşmeyi yüzde olarak ifade et.
- **Tip 5 — Load balancing denklemi:** $D_2 = D_3$ koşulunu kur, $V_2S_2 = V_3S_3$ denklemini çöz.

=Main Takeaway — Her Şeyin Özeti

Performance bounds, bir sistemi tam olarak çözmeden önce maksimum kapasitesini ve davranışını anlamamızı sağlar. Sistemin maksimum throughput'u D_{max} tarafından sınırlanır. N^* kullanıcısına kadar daha fazla kullanıcı eklemek throughput'u artırır. Bottleneck olmayan kaynağa yatırım yapmak israftır.